

ZingyBank

Full Retail Banking Platform

Production-grade microservices banking system
Java 21 · Spring Boot 3.4 · React 18 · Kubernetes · Terraform

Technical Project Overview | April 2025

Table of Contents

1. Executive Summary
2. System Architecture
3. Microservices
4. Frontend Application
5. Data Layer
6. Security & Compliance
7. Observability Stack
8. DevOps & CI/CD Pipeline
9. Kubernetes & Helm
10. Infrastructure as Code (Terraform)
11. Local Development
12. Screenshots
13. Appendix — Credentials & URLs

1. Executive Summary

ZingyBank is a production-grade, cloud-agnostic retail banking platform built to demonstrate enterprise-level software engineering practices. The system implements a full suite of retail banking capabilities — customer onboarding, account management, fund transfers, bill payments, loan origination, card issuance, KYC/AML compliance, and immutable audit logging — across 11 independently deployable microservices.

The platform is designed to run on any cloud (Azure, AWS, GCP) or on-premises Kubernetes cluster with no code changes. It ships with complete Infrastructure as Code (Terraform), GitOps delivery (ArgoCD), a multi-cloud CI/CD pipeline (GitHub Actions), and a full observability stack (Prometheus, Grafana, Jaeger, Loki).

['Microservices', 'Frontend', 'Databases', 'Cloud Targets', 'K8s Namespaces']				
11	React 18 + TypeScript	10 (per-service)	Azure · AWS · GCP · Any K8s	3

2. System Architecture

The platform follows a microservices architecture pattern with a single entry point (API Gateway), domain-isolated services, and asynchronous event-driven communication via Apache Kafka. Each service owns its own PostgreSQL database — enforcing domain isolation and independent deployability.

2.1 Request Flow

External traffic enters through an NGINX Ingress Controller (Kubernetes) or directly to the API Gateway (local dev). Spring Cloud Gateway MVC routes requests to downstream services based on path prefixes. Each downstream service validates JWT tokens independently.

- Browser / Mobile → NGINX Ingress (TLS termination, rate limit, security headers)
- NGINX Ingress → React Frontend (static assets via nginx:alpine)
- Browser API calls → NGINX Ingress → API Gateway (port 8080)
- API Gateway → microservice (JWT forwarded, Resilience4j applied)
- Microservice → PostgreSQL (via PgBouncer connection pooler)
- Microservice → Kafka (async domain events to other services)
- Microservice → Redis (session cache, rate limiting, idempotency keys)
- Microservice → HashiCorp Vault (secrets at runtime via environment injection)

2.2 Technology Stack

Category	Technology
Language	Java 21 (LTS)
Framework	Spring Boot 3.4, Spring Cloud 2024.0
Build	Maven 3.9 (multi-module, parallel builds)
Frontend	React 18, TypeScript, Vite, TailwindCSS v4, Zustand, React Query
Database	PostgreSQL 16, Flyway migrations, database-per-service
Connection Pool	PgBouncer (transaction mode, 1000 max connections)
Cache	Redis 7 (session store, rate limiting, idempotency)
Messaging	Apache Kafka (Confluent 7.7.1, event-driven async)
Auth	Spring Security, JWT (jjwt), BCrypt-12, RBAC
Resilience	Resilience4j (circuit breaker, retry, rate limiter, time limiter)
API Docs	SpringDoc OpenAPI 2 (Swagger UI per service)
Tracing	OpenTelemetry SDK → Jaeger (OTLP HTTP/gRPC)
Monitoring	Prometheus, Grafana, Micrometer
Logging	Loki, Promtail, SLF4J/Logback (structured JSON)
Secrets	HashiCorp Vault + External Secrets Operator (K8s)
Service Mesh	Linkerd (mTLS, retries, timeouts, per-route policies)

Category	Technology
Container	Docker (multi-stage builds), Docker Compose (local dev)
Orchestration	Kubernetes 1.32, Helm 3, ArgoCD (GitOps)
IaC	Terraform >= 1.5 (Azure AKS + AWS EKS modules)

3. Microservices

Each microservice is a self-contained Spring Boot application with its own database, Flyway migration scripts, Resilience4j configuration, OpenTelemetry tracing, and Prometheus metrics endpoint. Services communicate synchronously via REST through the API Gateway, and asynchronously via Kafka domain events.

Service	Port	Responsibility
api-gateway	8080	Spring Cloud Gateway MVC — routing, rate limiting, JWT forwarding to all services
auth-service	8081	JWT authentication, BCrypt-12, RBAC (5 roles), MFA, account lockout, refresh tokens
account-service	8082	Account CRUD, balance management, CHECKING / SAVINGS / BUSINESS account types
transaction-service	8083	Fund transfers, deposits, withdrawals, event sourcing, idempotency
payment-service	8084	Bill payments, scheduled/recurring payments, payment history
loan-service	8085	Loan applications, approval workflows, amortization schedule generation
card-service	8086	Virtual/physical card issuance, activation, limit management, block/report stolen
kyc-service	8087	KYC/AML verification, document upload, sanctions screening, PEP checks
notification-service	8088	Email/SMS/push notifications — event-driven via Kafka, SMTP via MailHog
statement-service	8089	Account statements, PDF generation, date-range filtering
audit-service	8090	Immutable append-only audit log with SHA-256 hash chain (tamper detection)

3.1 Resilience Configuration (per service)

Every service applies Resilience4j patterns to prevent cascading failures:

- Circuit Breaker — opens after 50% failure rate over 10 requests; waits 30s before half-open
- Retry — max 3 attempts with 500ms exponential backoff
- Rate Limiter — 100 requests/second per instance
- Time Limiter — 5s timeout on all downstream calls

3.2 RBAC — User Roles

Role	Assigned To	Access
CUSTOMER	Default on registration	Accounts, transactions, cards, loans, KYC, statements
TELLER	Bank teller staff	All customer data + manual transaction overrides
MANAGER	Branch manager	Teller access + loan approvals + Observability dashboard
COMPLIANCE_OFFICER	AML/KYC reviewer	KYC workflows, suspicious activity reports + Observability

Role	Assigned To	Access
ADMIN	System administrator	Full system access including all observability tooling

4. Frontend Application

The frontend is a single-page React 18 application written in TypeScript, built with Vite, and styled with TailwindCSS v4. It communicates with the API Gateway via REST and ships OpenTelemetry browser tracing and Loki log shipping built in.

Concern	Implementation
Framework	React 18 + TypeScript + Vite
Styling	TailwindCSS v4 (utility-first, custom ZingyBank design tokens)
State	Zustand (global stores: auth, UI, notifications)
Data Fetching	React Query (server-state, caching, optimistic updates)
Forms	react-hook-form + Zod (schema validation)
Routing	React Router v6 (code-split lazy routes)
Tracing	OpenTelemetry browser SDK — traces to Jaeger via /otlp/ nginx proxy
Log Shipping	Structured JSON logs to Loki via /loki/ nginx proxy
Serving	nginx:alpine (non-root, read-only FS, gzip, security headers)
Build	Multi-stage Dockerfile (node:20-alpine builder → nginx:alpine)

4.1 Pages & Features

Page	Features
Dashboard	Balance overview, balance trend chart, recent transactions, quick-action cards
Accounts	Account list, create account, account detail with transaction history
Transfer	Fund transfer form with account selector, amount, description
Payments	New payment form, payment history tab
Loans	Loan application form, active loans list, amortization view
Cards	Card management — issue, activate, set limits, block/report
Statements	Statement download with date-range filter
Profile	User profile, personal info, security settings
Observability	Admin-only — embedded Grafana dashboards, links to Jaeger / Kafka UI / MailHog

5. Data Layer

5.1 Database per Service

Each microservice owns exactly one PostgreSQL 16 database. No cross-database joins are permitted. Cross-domain data needs are resolved through service APIs or Kafka events. Flyway migrations run at service startup.

Database	Owner Service	Key Tables
zingybank_auth	auth-service	users, user_roles, refresh_tokens
zingybank_account	account-service	accounts, account_balances
zingybank_transaction	transaction-service	transactions, transaction_events
zingybank_payment	payment-service	payments, payment_schedules
zingybank_loan	loan-service	loan_applications, loans, amortization_schedules
zingybank_card	card-service	cards, card_transactions, card_limits
zingybank_kyc	kyc-service	kyc_records, documents, verification_results
zingybank_notification	notification-service	notifications, notification_templates
zingybank_statement	statement-service	statements, statement_items
zingybank_audit	audit-service	audit_events (append-only, SHA-256 hash chain)

5.2 PgBouncer Connection Pooling

- Transaction pool mode — lowest latency, highest throughput
- 1000 max client connections mapped to 100 server connections
- Services connect to pgbouncer:5432 (K8s) or localhost:5433 (local)
- 2 replicas in Kubernetes with PodDisruptionBudget (minAvailable: 1)

5.3 Redis

- Session store — JWT refresh token metadata
- Rate limiting — sliding window counters per IP and user
- Idempotency keys — prevent duplicate financial transactions
- 256MB max memory with LRU eviction policy

6. Security & Compliance

6.1 Authentication & Authorisation

- JWT tokens — 15-minute access tokens, 7-day refresh tokens
- BCrypt cost factor 12 for password hashing
- 5 RBAC roles — CUSTOMER, TELLER, MANAGER, COMPLIANCE_OFFICER, ADMIN
- Account lockout after repeated failed login attempts
- MFA-ready (TOTP secret stored per user)

6.2 PCI DSS Compliance Features

Requirement	Category	Implementation
PCI DSS Req 1	Network segmentation	Kubernetes NetworkPolicies — default-deny-all, microsegmentation per service
PCI DSS Req 3	Data protection	Encryption at rest (AES-256), card numbers stored encrypted, PII field-level encryption
PCI DSS Req 4	Encryption in transit	TLS 1.3 enforced, HSTS headers, Linkerd mTLS between all services
PCI DSS Req 7	Access control	RBAC, Pod Security Standards (restricted), non-root containers
PCI DSS Req 10	Audit trail	Immutable append-only audit log with SHA-256 hash chain (tamper detection)
PCI DSS Req 11	Vulnerability management	Trivy image scanning in CI, SonarQube SAST, OWASP dependency checks
KYC/AML	Identity verification	Document upload, sanctions screening, PEP checks, suspicious activity reporting

6.3 Secrets Management

- HashiCorp Vault — KV v2 secrets engine, all credentials stored centrally
- Local dev — Vault in dev mode (token: zingybank-local-vault-token)
- Kubernetes — External Secrets Operator syncs Vault → K8s Secrets (1h refresh)
- Zero secrets in source code, Docker images, or environment files
- Azure Key Vault integration via AKS Workload Identity in cloud deployments

7. Observability Stack

ZingyBank implements the three pillars of observability — metrics, traces, and logs — with full correlation between them. Frontend browser sessions also emit traces and structured logs to the same backend.

Tool	Port	Role
Prometheus	9090	Scrapes /actuator/prometheus from all 11 services every 15s
Grafana	3001	ZingyBank Service Overview dashboard + JVM, Kafka, PostgreSQL dashboards
Jaeger	16686	Distributed tracing via OTLP. 100% sampling in dev. Trace-to-log correlation
Loki	3100	Log aggregation. Promtail ships container logs. Browser logs via nginx proxy
Promtail	—	Tails Docker container logs and ships structured JSON to Loki
OpenTelemetry	—	Browser SDK instruments all fetch/XHR calls. Traces proxied via /otlp/

7.1 Grafana — ZingyBank Service Overview Dashboard

Auto-provisioned via config files at infrastructure/grafana/provisioning/:

- Services Up — count of healthy service instances
- Total HTTP Requests (5m) — request volume stat panel
- Error Rate (5m) — percentage of 5xx responses
- P99 Latency — 99th percentile response time
- Active JVM Threads — thread pool health
- HTTP Request Rate by Service — timeseries per service
- HTTP Error Rate by Service — timeseries per service
- JVM Heap Usage — timeseries per service
- P95 Latency — timeseries per service
- Frontend Error Logs — Loki log panel
- All Service Logs — Loki log panel with label filters

8. DevOps & CI/CD Pipeline

The pipeline is implemented in GitHub Actions with two workflows: CI (ci.yml) and CD (cd.yml). Every push to main triggers CI; on success, CD automatically deploys to staging and waits for manual approval before promoting to production.

8.1 CI Pipeline — ci.yml

Stage	Details
Build & Test (Java)	mvn clean verify — compiles all 11 services in parallel (-T4), runs JUnit tests
Build & Test (React)	npm ci + tsc --noEmit + npm run build — type check and production build
Security Scan	Trivy image vulnerability scan, SonarQube SAST (optional)
Build & Push Images	Multi-arch Docker build pushed to GHCR (always) + ACR/ECR/GCP AR (optional)

8.2 CD Pipeline — cd.yml

Step	Details
Trigger	workflow_run on CI success — only runs when CI passes
Staging	Auto — updates helm/values/staging/.yaml image tags, commits, pushes
ArgoCD sync	ArgoCD detects the git push and auto-syncs the staging namespace
Production	Requires manual approval in GitHub Environments (protected branch)
Release	GitHub Release created automatically with image tag and commit SHA

8.3 Multi-Cloud Image Registry Strategy

Registry	Status	Activation
GHCR	Always active	Built-in GITHUB_TOKEN — no secrets needed
Azure ACR	Optional	Set ACR_USERNAME + ACR_PASSWORD secrets
AWS ECR	Optional	Set AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY + AWS_ACCOUNT_ID
GCP AR	Optional	Set GCP_SA_KEY + GCP_PROJECT_ID secrets

9. Kubernetes & Helm

9.1 Namespaces

Namespace	Contents
zingybank	Restricted pod security standards. All 11 microservices + frontend + PgBouncer
zingybank-monitoring	Prometheus, Grafana, Alertmanager, Loki, Jaeger
zingybank-ingress	NGINX Ingress Controller
argocd	ArgoCD GitOps controller

9.2 Helm Base Chart — base-service

A single shared Helm chart (infrastructure/helm/charts/base-service) is used for all 11 microservices. Per-service overrides are in helm/values/.yaml.

- Deployment with configurable replicas, image, env vars, and secret injection
- ClusterIP Service
- HPA — 2–10 replicas, 70% CPU / 80% memory targets
- PodDisruptionBudget — minAvailable: 1
- Non-root user (UID 1000), read-only root filesystem, no privilege escalation
- Liveness probe — /actuator/health/liveness (45s initial delay)
- Readiness probe — /actuator/health/readiness (30s initial delay)
- Pod anti-affinity — prefer spreading across nodes
- Node selector — nodepool-type: application

9.3 ArgoCD Sync Waves (startup order)

Wave	Services
Wave 1	api-gateway, frontend
Wave 2	auth-service
Wave 3	account-service, transaction-service, payment-service, loan-service, card-service, kyc-service
Wave 4	notification-service, statement-service, audit-service

9.4 Network Security (PCI DSS)

- Default-deny-all NetworkPolicy — all ingress and egress blocked by default
- allow-dns — UDP/TCP port 53 egress for all pods
- allow-gateway-to-services — API Gateway → all microservices (ports 8080–8090)
- allow-services-to-kafka — all services → Kafka (port 9092)
- allow-prometheus-scrape — Prometheus → all pods (metrics ports)
- allow-ingress-to-gateway — NGINX Ingress → API Gateway + Frontend
- allow-jobs-to-services — seed/admin K8s Jobs → auth-service

10. Infrastructure as Code (Terraform)

All cloud infrastructure is managed by Terraform ≥ 1.5 . Azure is the primary cloud (AKS, PostgreSQL Flexible Server, ACR, Key Vault). AWS modules provide a disaster recovery option (EKS, RDS, ECR). Remote state is stored in Azure Blob Storage.

10.1 Azure Modules

Module	Provisions
azure/networking	VNet, 4 subnets (system/app/DB/private-endpoints), NSGs, PostgreSQL private DNS zone
azure/aks	Private AKS 1.32 cluster, system + app node pools, AAD RBAC, Log Analytics, Key Vault CSI
azure/database	PostgreSQL 16 Flexible Server, 10 databases, private endpoint (no public access)
azure/acr	Premium ACR with geo-replication, private endpoint
azure/keyvault	HSM-backed Key Vault, AKS managed identity access policy, soft-delete disabled (dev)

10.2 AWS Modules (DR)

Module	Provisions
aws/networking	VPC, 3-AZ public/private subnets, NAT gateways per-AZ, EKS + RDS security groups
aws/eks	Private EKS cluster, system + app node groups, IRSA OIDC provider, KMS encryption
aws/rds	Multi-AZ RDS PostgreSQL 16, KMS encryption, 35-day backups, enhanced monitoring
aws/ecr	11 ECR repos (one per service), scan-on-push, lifecycle policies, immutable tags in prod

10.3 Environments

Env	Cloud	VNet CIDR	K8s	Resource Group / Region	State Key
dev	Azure	10.0.0.0/16	1.32	zingybank-dev-rg	dev.terraform.tfstate
staging	Azure	10.1.0.0/16	1.32	zingybank-staging-rg	staging.terraform.tfstate
prod	Azure	10.2.0.0/16	1.32	zingybank-prod-rg	prod.terraform.tfstate
prod-dr	AWS	10.20.0.0/16	1.32	us-east-1	prod-dr.terraform.tfstate

11. Local Development

The full stack — all 11 microservices, PostgreSQL, Redis, Kafka, Vault, Grafana, Prometheus, Jaeger, Loki, Promtail, PgBouncer, MailHog, Kafka UI, and the React frontend — runs locally via Docker Compose with a single command.

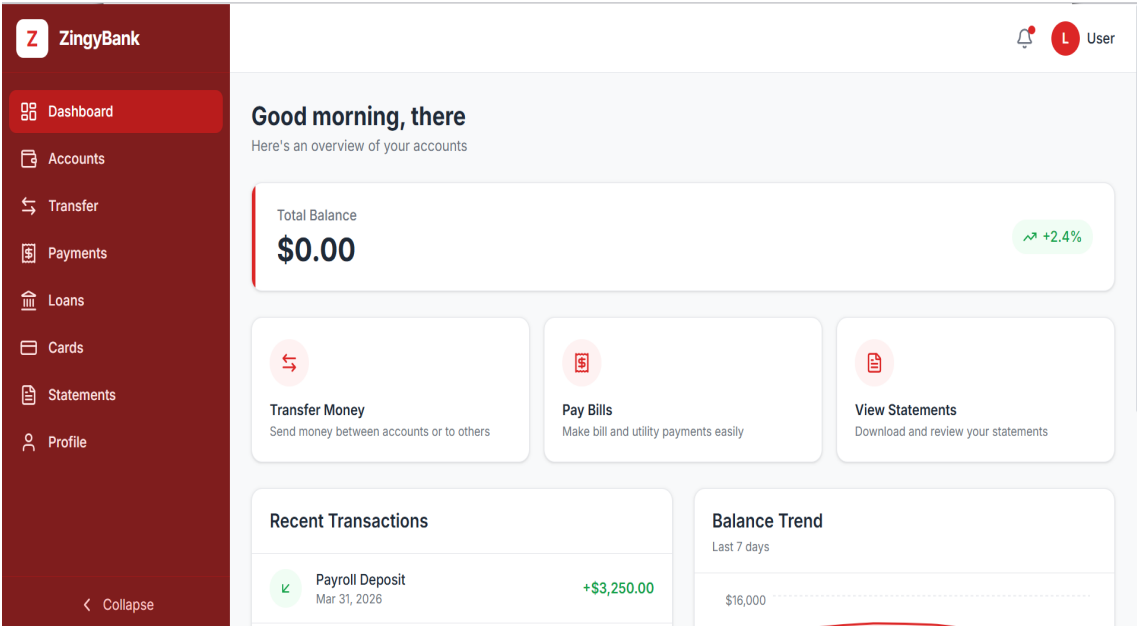
11.1 Quick Start

```
git clone https://github.com/LycanTech/ZingyBank.git && cd ZingyBank
docker compose up -d postgres redis zookeeper kafka pgbouncer mailhog kafka-ui
docker compose up -d prometheus jaeger loki promtail grafana vault
docker compose up -d # starts all 11 microservices + frontend
```

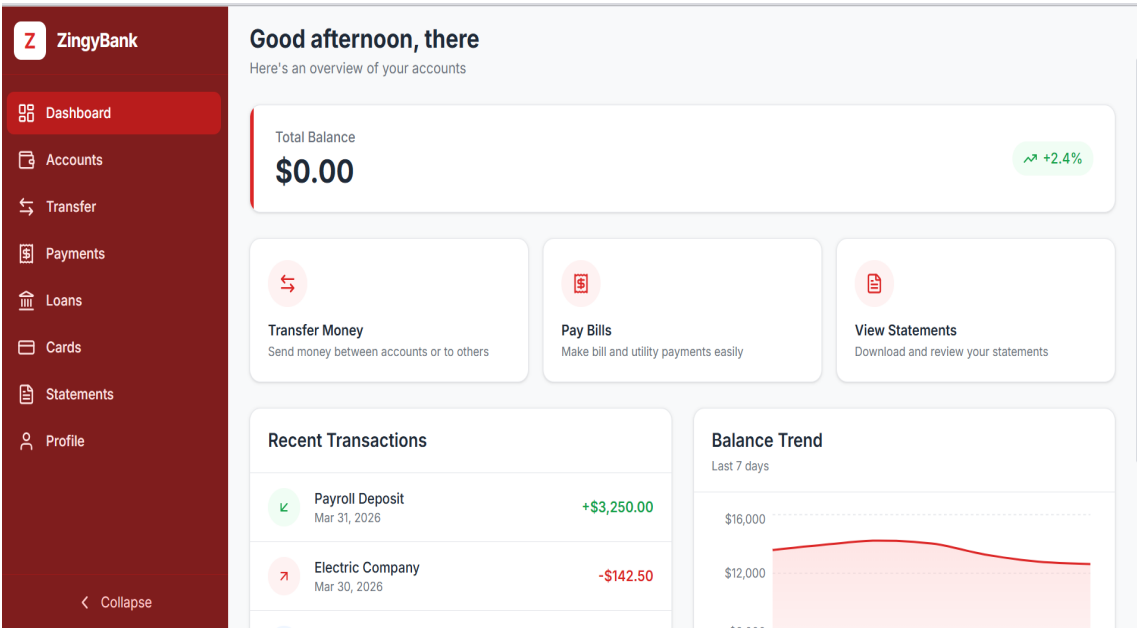
11.2 Local URLs

Service	URL	Credentials / Notes
Frontend	http://localhost:3002	Register to get started
API Gateway	http://localhost:8080	Entry point for all API calls
Grafana	http://localhost:3001	admin / Grafana@Admin2024!
Prometheus	http://localhost:9090	No auth
Jaeger	http://localhost:16686	No auth
Kafka UI	http://localhost:9091	No auth
MailHog	http://localhost:8025	No auth
Vault	http://localhost:8200/ui	Token: zingybank-local-vault-token
auth-service Swagger	http://localhost:8081/swagger-ui.html	Direct service access
account-service Swagger	http://localhost:8082/swagger-ui.html	Direct service access

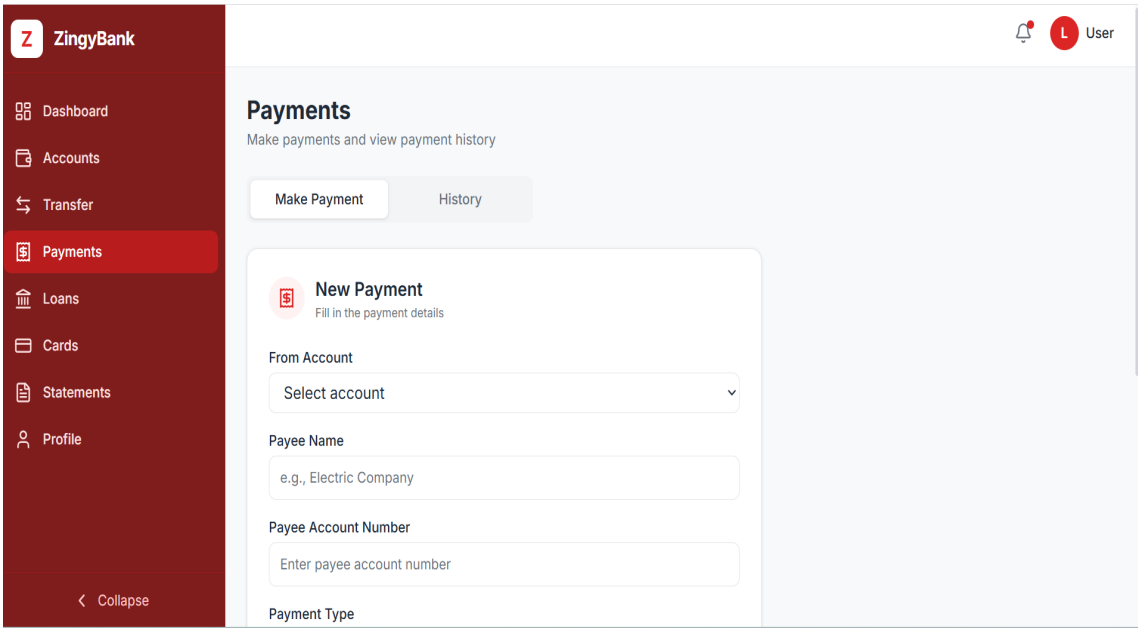
12. Screenshots



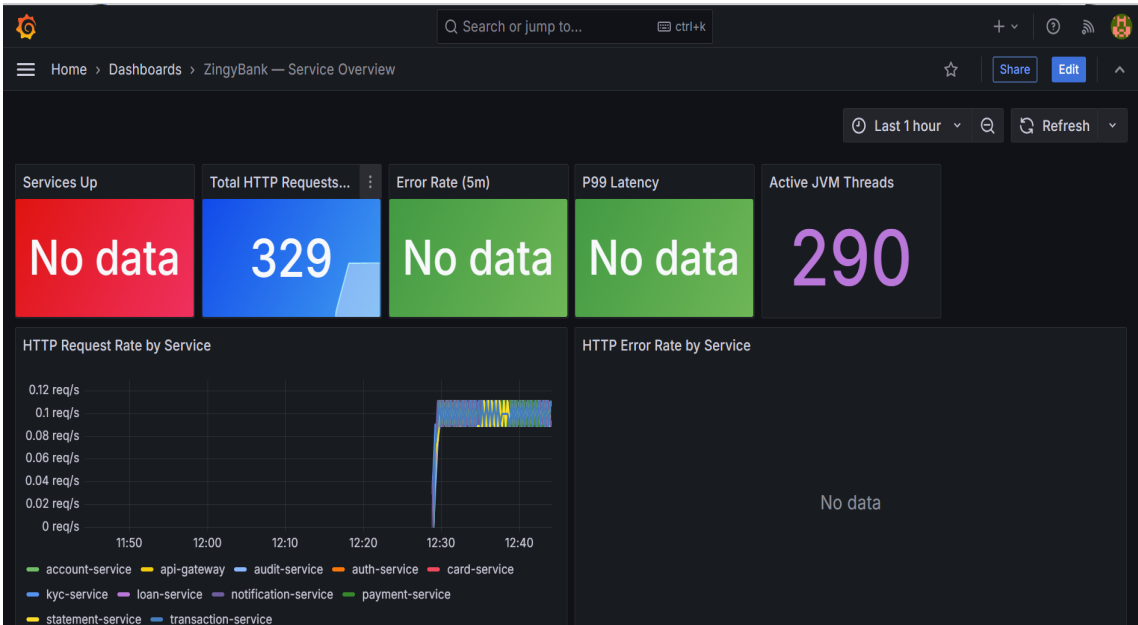
Login Page — JWT authentication with email and password



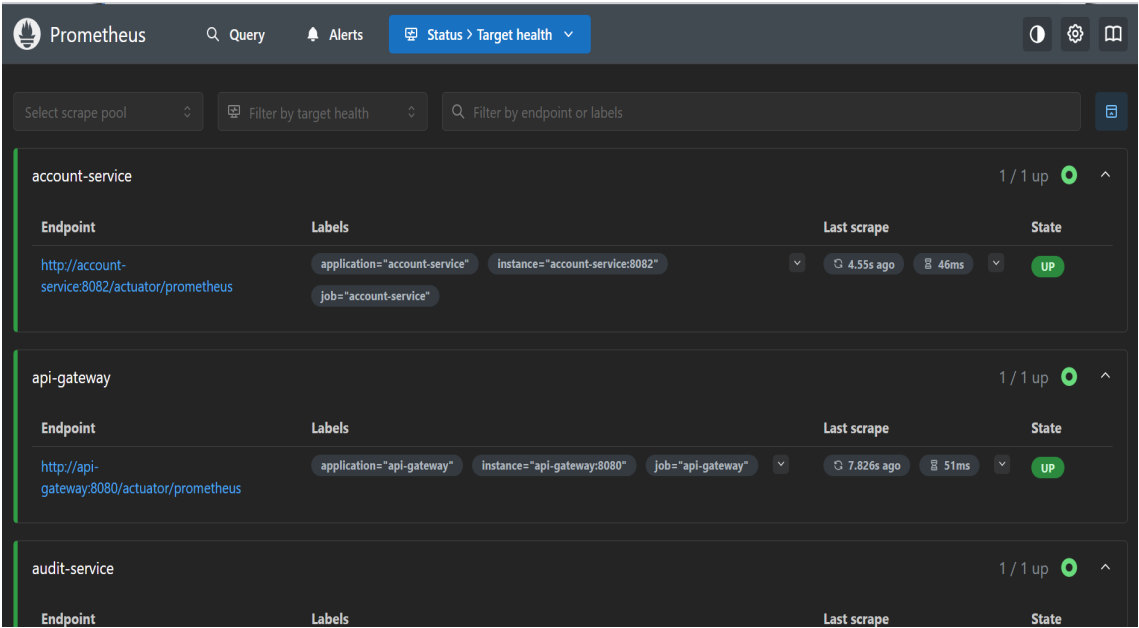
Dashboard — Balance overview, balance trend, recent transactions



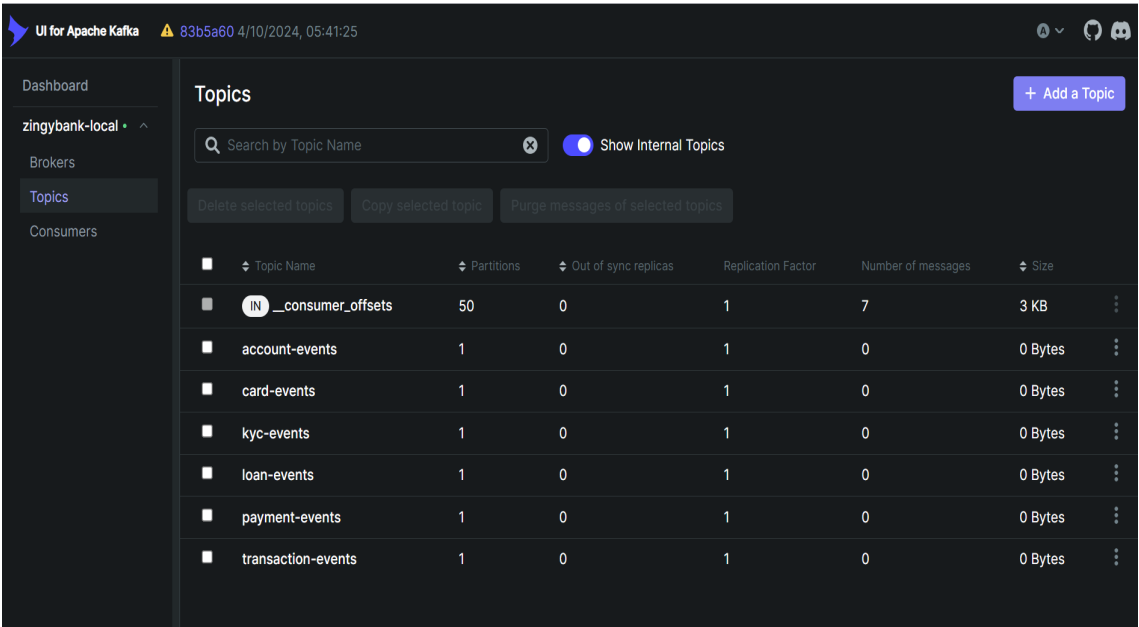
Payments Page — New payment form and payment history



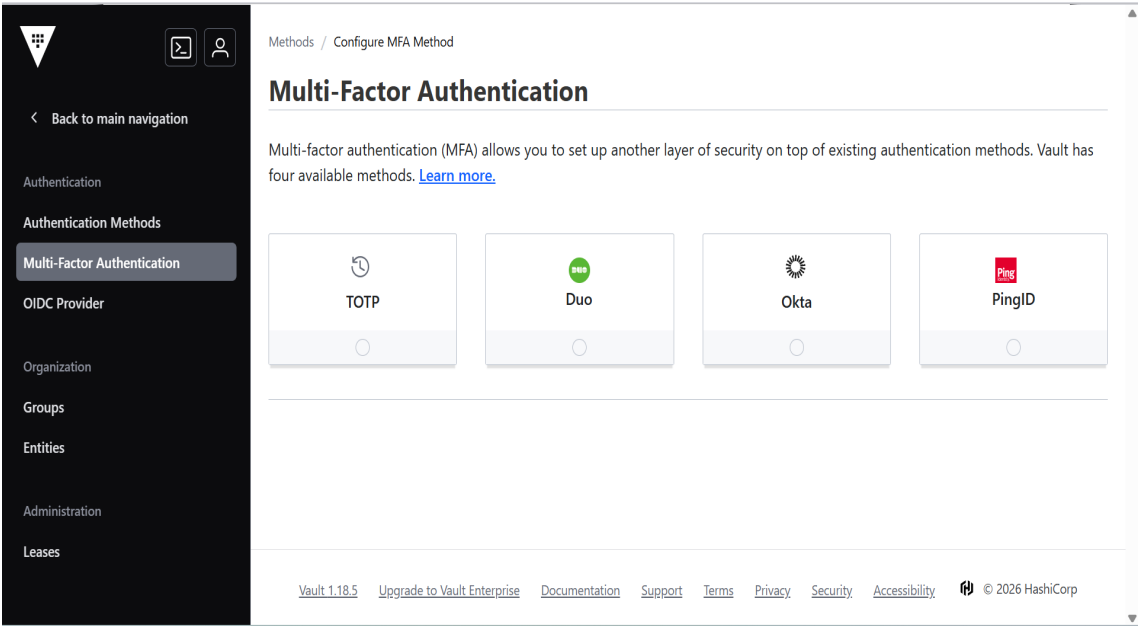
Grafana — ZingyBank Service Overview Dashboard



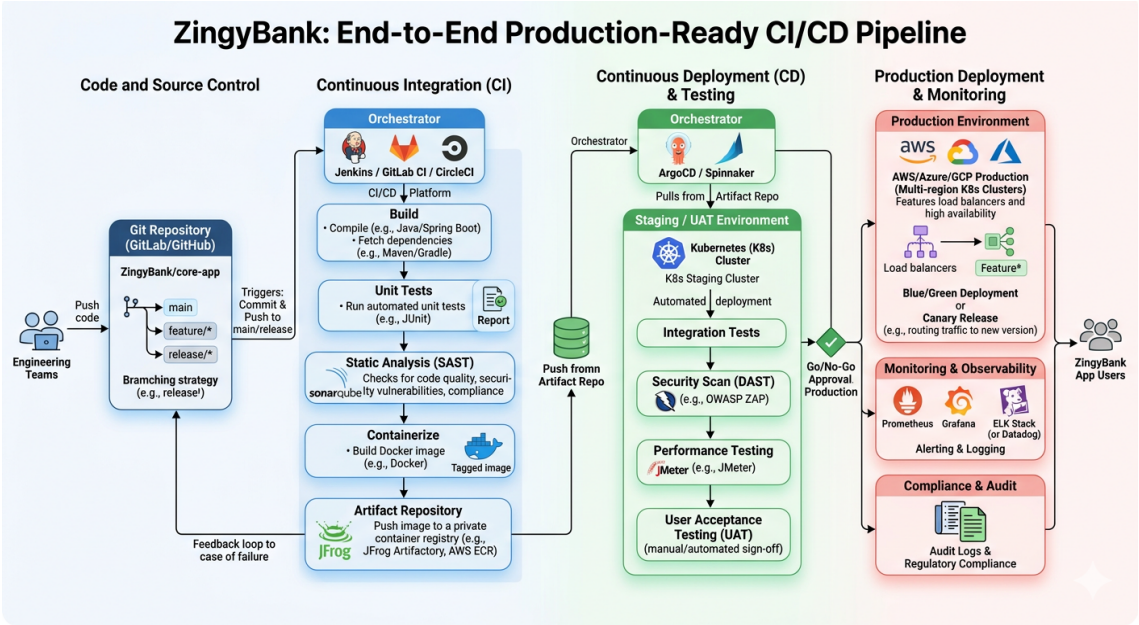
Prometheus — All 11 service scrape targets active



Kafka UI — Topic and message inspection



HashiCorp Vault — Secrets management UI



CI/CD Pipeline — End-to-end build, test, scan, deploy

13. Appendix — Credentials & URLs

■■ Local development credentials only. Never use in production.

Service	Host:Port	Username	Password / Token	Notes
PostgreSQL	localhost:5432	zingybank	ZingyDev2024	Direct DB access
PgBouncer	localhost:5433	zingybank	ZingyDev2024	Connection pooler
Redis	localhost:6379	—	Chikwex@ZingyDev!	Cache/session store
Grafana	localhost:3001	admin	Grafana@Admin2024!	Monitoring UI
Vault	localhost:8200	—	zingybank-local-vault-token	Secrets manager

Test Accounts (after running seed job)

Email	Password	Role	Use For
admin@zingybank.test	TestAdmin123!	ADMIN	Full system access
teller@zingybank.test	TestTeller123!	TELLER	Transaction operations
alice@zingybank.test	TestAlice123!	CUSTOMER	End-to-end account testing
bob@zingybank.test	TestBob123!	CUSTOMER	Transfer testing (Bob ↔ Alice)

Git Repositories

GitHub: <https://github.com/LycanTech/ZingyBank>

Azure Repos: https://dev.azure.com/chikweazinge/ZingyBank/_git/ZingyBank